

University of the People

CS-1102: Programming 1

The `Function` Interface and Streams in Java

Andreas Dau

Student ID: C644260

Programming Assignment, Unit 8

June 4, 2026

Background

Java 8 introduced functional interfaces and the Stream API to allow developers to express data transformations more directly (Liang, 2020). The `Function<T, R>` interface represents any operation that takes one input of type `T` and produces one output of type `R`. Its single abstract method, `apply(T t)`, can be implemented with a lambda expression, making it possible to store, pass, and combine functions as values (Urma, Fusco, & Mycroft, 2018). The related `Predicate<T>` interface represents a boolean-valued function and supports composition through `and()`, `or()`, and `negate()`. Streams build on these interfaces: a stream pipeline chains intermediate operations such as `filter()` and `map()`, which accept functional interfaces as arguments, and defers execution until a terminal operation such as `toList()` or `average()` is called (Goetz et al., 2006). This lazy evaluation model keeps memory consumption bounded even for large datasets.

Program Description

The program is a command-line tool that processes a dataset of company employees. Each employee has four attributes: name, age, department, and salary. The tool accepts one or more `--filter` expressions at runtime, builds a composite `Predicate<Employee>` from them, and passes it to a stream pipeline that produces a filtered list of names and departments alongside the average salary of the matching employees.

The `Function` Interface in Use

A `Function<Employee, String>` is defined to concatenate each employee's name and department into a single string. This function is passed directly to `Stream.map()`, which applies it to every element in the filtered stream:

```
1      Function<Employee, String> nameAndDept =  
2      emp -> emp.getName() + ", " + emp.getDepartment();  
3  
4      List<String> result = employees.stream()  
5      .filter(combinedFilter)  
6      .map(nameAndDept)  
7      .toList();  
8
```

Filter Expressions and Predicate Composition

The `FilterParser` class parses a filter string such as `age>30` or `department=Engineering` into a `Predicate<Employee>`. The supported operators are `=`, `~` (contains), `>`, `<`, `>=`, and `<=`. When multiple `--filter` flags are given, the predicates are combined with `Predicate.and()`, so all conditions must match:

```

1 Predicate<Employee> combined = emp -> true;
2 for (int i = 0; i < args.length - 1; i++) {
3     if ("--filter".equals(args[i])) {
4         combined = combined.and(FilterParser.parse(args[i + 1]));
5     }
6 }
7

```

Average Salary

The average salary is computed using `mapToDouble()` to produce a primitive `DoubleStream`, followed by the built-in `average()` reduction. The result is wrapped in `OptionalDouble` to handle an empty dataset without throwing an exception:

```

1 OptionalDouble avgSalary = employees.stream()
2     .filter(combinedFilter)
3     .mapToDouble(Employee::getSalary)
4     .average();
5
6 avgSalary.ifPresentOrElse(
7     avg -> System.out.printf("$%.2f\n", avg),
8     () -> System.out.println("(no data)");
9

```

CSV Support and EmployeeFactory

The `-i/--infile` flag loads a CSV file instead of the built-in dataset. The companion tool `EmployeeFactory` generates CSV files of arbitrary size for testing:

```

java EmployeeFactory 1000000 big_dataset.csv
java EmployeeProcessor -i big_dataset.csv --filter age>30 --filter
salary>=70000

```

The stream pipeline requires no modification for larger datasets because elements are processed one at a time through lazy evaluation.

Design Decision: Class Instead of Record

The `Employee` data model is implemented as a static inner class rather than a Java record. While records, stable since Java 16, would generate the constructor and accessors automatically, the explicit class form was chosen to keep the structure transparent and consistent with the course material.

Source Code

`Employee.java`

```

1  /**
2   * Represents an employee in the dataset.
3   * Intentionally implemented as a static inner class rather than a record,
4   * to maintain compatibility with the course's Java version requirements
5   * and to demonstrate explicit OOP design.
6   */
7  public class Employee {
8
9      private final String name;
10     private final int    age;
11     private final String department;
12     private final double salary;
13
14     public Employee(String name, int age, String department, double salary)
15     {
16         this.name      = name;
17         this.age       = age;
18         this.department = department;
19         this.salary    = salary;
20     }
21
22     public String getName()      { return name; }
23     public int    getAge()       { return age; }
24     public String getDepartment() { return department; }
25     public double getSalary()    { return salary; }
26
27     @Override
28     public String toString() {
29         return String.format("%-12s %-15s age=%-3d salary=$%.0f",
30                               name, department, age, salary);
31     }
32 }

```

FilterParser.java

```

1  import java.util.function.Predicate;
2
3  /**
4   * Parses a filter expression string into a Predicate<Employee>.
5   * Supported syntax: field operator value
6   *
7   * Fields:    name, age, department, salary
8   * Operators: =    exact match (strings and numbers)
9   *             ~    contains, case-insensitive (strings only)
10  *             >    greater than (numbers only)
11  *             <    less than (numbers only)
12  *             >=   greater than or equal (numbers only)
13  *             <=   less than or equal (numbers only)
14  *
15  * Examples:
16  *   age>30
17  *   salary>=70000
18  *   department=Engineering
19  *   name~ali
20  */
21  public class FilterParser {
22
23      /**
24       * Parses a single filter expression and returns the corresponding
25       * Predicate.
26       *
27       * @param expression e.g. "age>30" or "department=Engineering"
28       * @return a Predicate<Employee> that evaluates the expression
29       * @throws IllegalArgumentException if the expression cannot be parsed
30       */
31      public static Predicate<Employee> parse(String expression) {
32
33          // Determine operator - check two-char operators first
34          String op;
35          if (expression.contains(">=")) op = ">=";
36          else if (expression.contains("<=")) op = "<=";
37          else if (expression.contains(">")) op = ">";
38          else if (expression.contains("<")) op = "<";
39          else if (expression.contains("~")) op = "~";
40          else if (expression.contains("=")) op = "=";
41          else throw new IllegalArgumentException(
42              "Unknown operator in filter expression: " + expression);
43
44          String[] parts = expression.split(op, 2);
45          if (parts.length != 2)
46              throw new IllegalArgumentException(
47                  "Invalid filter expression: " + expression);
48
49          String field = parts[0].trim().toLowerCase();
50          String value = parts[1].trim();
51
52          return switch (field) {
53              case "age" -> numericPredicate(op, parseNumber(value, "
age"),
                    emp -> (double) emp.getAge());

```

```

54         case "salary"      -> numericPredicate(op, parseNumber(value, "
salary"),
55             emp -> emp.getSalary());
56         case "name"         -> stringPredicate(op, value,
57             emp -> emp.getName());
58         case "department" -> stringPredicate(op, value,
59             emp -> emp.getDepartment());
60         default -> throw new IllegalArgumentException(
61             "Unknown field: '" + field +
62             "'. Valid fields: name, age, department, salary");
63     };
64 }
65
66 // --- helpers ---
67
68 private static double parseNumber(String value, String field) {
69     try {
70         return Double.parseDouble(value);
71     } catch (NumberFormatException e) {
72         throw new IllegalArgumentException(
73             "'" + value + "' is not a valid number for field '" + field
74             + "'");
75     }
76 }
77
78 @FunctionalInterface
79 private interface EmployeeToDouble {
80     double apply(Employee e);
81 }
82
83 @FunctionalInterface
84 private interface EmployeeToString {
85     String apply(Employee e);
86 }
87
88 private static Predicate<Employee> numericPredicate(
89     String op, double threshold, EmployeeToDouble extractor) {
90     return switch (op) {
91         case ">" -> emp -> extractor.apply(emp) > threshold;
92         case "<" -> emp -> extractor.apply(emp) < threshold;
93         case ">=" -> emp -> extractor.apply(emp) >= threshold;
94         case "<=" -> emp -> extractor.apply(emp) <= threshold;
95         case "=" -> emp -> extractor.apply(emp) == threshold;
96         default -> throw new IllegalArgumentException(
97             "Operator '" + op + "' is not supported for numeric fields"
98     );
99 }
100
101 private static Predicate<Employee> stringPredicate(
102     String op, String value, EmployeeToString extractor) {
103     return switch (op) {
104         case "=" -> emp -> extractor.apply(emp)
105             .equalsIgnoreCase(value);
106         case "~" -> emp -> extractor.apply(emp)
107             .toLowerCase()
108             .contains(value.toLowerCase());
109         default -> throw new IllegalArgumentException(

```

```

109         "Operator '" + op + "' is not supported for string fields")
110     };
111 }
112 }

```

CSVReader.java

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4  import java.util.ArrayList;
5  import java.util.List;
6
7  /**
8   * Reads a CSV file produced by EmployeeFactory and returns a list of
9   * Employee objects.
10   *
11   * Expected CSV format (with header row):
12   *   name,age,department,salary
13   *   Emp_a3f9,35,Engineering,90000.00
14   */
15  public class CSVReader {
16
17      /**
18       * Reads the given CSV file and returns its contents as a list of
19       * Employee objects.
20       * The first row is treated as a header and skipped.
21       *
22       * @param filePath path to the CSV file
23       * @return list of Employee objects parsed from the file
24       * @throws IOException if the file cannot be read
25       * @throws IllegalArgumentException if a row cannot be parsed
26       */
27      public static List<Employee> read(String filePath) throws IOException {
28          List<Employee> employees = new ArrayList<>();
29
30          try (BufferedReader reader = new BufferedReader(new FileReader(
31              filePath))) {
32              String line = reader.readLine(); // skip header
33
34              if (line == null) {
35                  throw new IllegalArgumentException(
36                      "File '" + filePath + "' is empty.");
37              }
38
39              int lineNumber = 1;
40              while ((line = reader.readLine()) != null) {
41                  lineNumber++;
42                  line = line.trim();
43                  if (line.isEmpty()) continue;
44
45                  String[] parts = line.split(",", 4);
46                  if (parts.length != 4) {
47                      throw new IllegalArgumentException(

```

```

45         "Invalid row at line " + lineNumber +
46         ": expected 4 columns, got " + parts.length +
47         " -> '" + line + "'");
48     }
49
50     try {
51         String name      = parts[0].trim();
52         int    age       = Integer.parseInt(parts[1].trim());
53         String department = parts[2].trim();
54         double salary    = Double.parseDouble(parts[3].trim())
55
56         ;
57         employees.add(new Employee(name, age, department,
58         salary));
59     } catch (NumberFormatException e) {
60         throw new IllegalArgumentException(
61             "Invalid number at line " + lineNumber +
62             ": " + e.getMessage() + " -> '" + line + "'");
63     }
64 }
65
66     return employees;
67 }
68 }

```

EmployeeFactory.java

```

1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4  import java.util.List;
5  import java.util.Random;
6
7  /**
8   * Generates a CSV file containing randomly created employee records.
9   *
10  * Usage:
11  *   java EmployeeFactory <count> <outputfile>
12  *
13  * Examples:
14  *   java EmployeeFactory 100 employees.csv
15  *   java EmployeeFactory 1000000 big_dataset.csv
16  *
17  * CSV format:
18  *   name,age,department,salary
19  */
20 public class EmployeeFactory {
21
22     private static final String USAGE = ""
23         Usage:
24             java EmployeeFactory <count> <outputfile>
25
26         Arguments:
27             count      Number of employee records to generate (must be > 0)
28             outputfile Path to the output CSV file

```



```

29
30     Examples:
31         java EmployeeFactory 100 employees.csv
32         java EmployeeFactory 1000000 big_dataset.csv
33     """;
34
35     private static final List<String> DEPARTMENTS =
36         List.of("Engineering", "Marketing", "HR", "Finance");
37
38     private static final int    MIN_AGE    = 18;
39     private static final int    MAX_AGE    = 65;
40     private static final double MIN_SALARY = 30_000;
41     private static final double MAX_SALARY = 150_000;
42
43     public static void main(String[] args) {
44
45         if (args.length != 2) {
46             System.err.println("Error: Expected exactly 2 arguments.");
47             System.err.print(USAGE);
48             System.exit(1);
49         }
50
51         int count;
52         try {
53             count = Integer.parseInt(args[0]);
54             if (count <= 0) throw new NumberFormatException();
55         } catch (NumberFormatException e) {
56             System.err.println("Error: count must be a positive integer,
got '" + args[0] + "'");
57             System.exit(1);
58             return;
59         }
60
61         String outputFile = args[1];
62
63         try {
64             generate(count, outputFile);
65             System.out.printf("Generated %,d records -> %s\n", count,
outputFile);
66         } catch (IOException e) {
67             System.err.println("Error writing file '" + outputFile + "': "
+ e.getMessage());
68             System.exit(1);
69         }
70     }
71
72     /**
73      * Generates {@code count} random employee records and writes them to a
CSV file.
74      *
75      * @param count      number of records to generate
76      * @param outputFile path to the output CSV file
77      * @throws IOException if the file cannot be written
78      */
79     public static void generate(int count, String outputFile) throws
IOException {
80         Random rng = new Random();
81

```

```

82     try (BufferedWriter writer = new BufferedWriter(new FileWriter(
outputFile))) {
83         writer.write("name,age,department,salary");
84         writer.newLine();
85
86         for (int i = 0; i < count; i++) {
87             String name      = "Emp_" + randomHex(rng, 4);
88             int    age       = MIN_AGE + rng.nextInt(MAX_AGE - MIN_AGE
+ 1);
89             String department = DEPARTMENTS.get(rng.nextInt(DEPARTMENTS
.size()));
90             double salary    = MIN_SALARY + rng.nextDouble() * (
MAX_SALARY - MIN_SALARY);
91
92             writer.write(String.format("%s,%d,%s,%.2f", name, age,
department, salary));
93             writer.newLine();
94         }
95     }
96 }
97
98 private static String randomHex(Random rng, int length) {
99     StringBuilder sb = new StringBuilder(length);
100     for (int i = 0; i < length; i++) {
101         sb.append(Integer.toHexString(rng.nextInt(16)));
102     }
103     return sb.toString();
104 }
105 }

```

EmployeeProcessor.java

```

1  import java.io.IOException;
2  import java.util.ArrayList;
3  import java.util.List;
4  import java.util.OptionalDouble;
5  import java.util.function.Function;
6  import java.util.function.Predicate;
7
8  /**
9   * CLI tool for processing an employee dataset using the Function interface
10  * and Java Streams.
11  *
12  * Usage:
13  *   java EmployeeProcessor [-i file] [--filter expression] ...
14  *
15  * Examples:
16  *   java EmployeeProcessor
17  *   java EmployeeProcessor -i employees.csv --filter age>30
18  *   java EmployeeProcessor --infile employees.csv --filter age>30 --filter
salary>=70000
19  */
20 public class EmployeeProcessor {
21
22     private static final String USAGE = ""

```

```

23      Usage:
24          java EmployeeProcessor [-i file] [--filter expression] ...
25
26      Options:
27          -h, --help            Show this help message
28          -?, --help            Same as above
29          -i, --infile file     Read employees from a CSV file instead of
30                                the built-in dataset
31          --filter expr         Filter employees by a field expression (
repeatable)
32
33      Filter expressions:
34          field operator value
35
36      Fields:      name, age, department, salary
37      Operators:  =    exact match (all fields)
38                  ~    contains, case-insensitive (name, department)
39                  >    greater than (age, salary)
40                  <    less than (age, salary)
41                  >=   greater than or equal (age, salary)
42                  <=   less than or equal (age, salary)
43
44      Multiple --filter flags are combined with AND logic.
45
46      Examples:
47          java EmployeeProcessor
48          java EmployeeProcessor -i employees.csv
49          java EmployeeProcessor -i employees.csv --filter age>30
50          java EmployeeProcessor -i employees.csv --filter age>30 --filter
salary>=70000
51          java EmployeeProcessor --filter department=Engineering
52          java EmployeeProcessor --filter name~al
53      """;
54
55      // Known flags that take a following value argument
56      private static final List<String> FLAGS_WITH_VALUE =
57          List.of("--filter", "-i", "--infile");
58
59      // Known flags that stand alone
60      private static final List<String> FLAGS_STANDALONE =
61          List.of("-h", "--help", "-?");
62
63      public static void main(String[] args) {
64
65          // Show help and exit if requested
66          for (String arg : args) {
67              if (FLAGS_STANDALONE.contains(arg)) {
68                  System.out.print(USAGE);
69                  return;
70              }
71          }
72
73          // Validate all flags and their arguments
74          for (int i = 0; i < args.length; i++) {
75              if (!args[i].startsWith("-")) continue;
76
77              if (!FLAGS_WITH_VALUE.contains(args[i]) && !FLAGS_STANDALONE.
contains(args[i])) {

```

```

78         System.err.println("Error: Unknown option '" + args[i] + "'
");
79         System.err.println("Run with -h or --help for usage
information.");
80         System.exit(1);
81     }
82
83     if (FLAGS_WITH_VALUE.contains(args[i])) {
84         if (i + 1 >= args.length || args[i + 1].startsWith("-")) {
85             System.err.println("Error: '" + args[i] + "' requires a
value.");
86             System.err.println("Run with -h or --help for usage
information.");
87             System.exit(1);
88         }
89     }
90 }
91
92 // 1. Load dataset: from CSV if --infile/-i given, otherwise use
built-in data
93 List<Employee> employees = loadEmployees(args);
94
95 // 2. Parse --filter arguments into Predicates and combine with AND
96 Predicate<Employee> combinedFilter = parseFilters(args);
97
98 // 3. Function interface: Employee -> "Name, Department"
99 Function<Employee, String> nameAndDept =
100     emp -> emp.getName() + ", " + emp.getDepartment();
101
102 // 4. Apply filter, then map to concatenated strings using streams
103 List<String> nameDeptList = employees.stream()
104     .filter(combinedFilter)
105     .map(nameAndDept)
106     .toList();
107
108 // 5. Average salary of filtered employees
109 OptionalDouble avgSalary = employees.stream()
110     .filter(combinedFilter)
111     .mapToDouble(Employee::getSalary)
112     .average();
113
114 // --- Output ---
115 System.out.println("=== Active filters ===");
116 printFilters(args);
117
118 System.out.println("\n=== Name, Department ===");
119 if (nameDeptList.isEmpty()) {
120     System.out.println("(no employees match the given filters)");
121 } else {
122     nameDeptList.forEach(System.out::println);
123 }
124
125 System.out.println("\n=== Average Salary ===");
126 avgSalary.ifPresentOrElse(
127     avg -> System.out.printf("%.2f\n", avg),
128     () -> System.out.println("(no data)");
129 }
130

```

```

131  /**
132   * Loads employees from a CSV file if -i/--infile is given,
133   * otherwise returns the built-in dataset.
134   */
135  private static List<Employee> loadEmployees(String[] args) {
136      for (int i = 0; i < args.length - 1; i++) {
137          if ("-i".equals(args[i]) || "--infile".equals(args[i])) {
138              String filePath = args[i + 1];
139              try {
140                  List<Employee> loaded = CSVReader.read(filePath);
141                  System.out.println("Loaded " + loaded.size() +
142                      " employees from " + filePath + "\n");
143                  return loaded;
144              } catch (IOException e) {
145                  System.err.println("Error reading file '" + filePath +
146                      "': " + e.getMessage());
147                  System.exit(1);
148              } catch (IllegalArgumentException e) {
149                  System.err.println("Error parsing file '" + filePath +
150                      "': " + e.getMessage());
151                  System.exit(1);
152              }
153          }
154      }
155
156      // Built-in dataset
157      List<Employee> employees = new ArrayList<>();
158      employees.add(new Employee("Alice", 35, "Engineering", 90_000));
159      employees.add(new Employee("Bob", 28, "Marketing", 55_000));
160      employees.add(new Employee("Carol", 42, "Engineering", 105_000));
161      employees.add(new Employee("David", 31, "HR", 62_000));
162      employees.add(new Employee("Eve", 24, "Marketing", 48_000));
163      employees.add(new Employee("Frank", 38, "Finance", 78_000));
164      return employees;
165  }
166
167  /**
168   * Reads --filter arguments from the CLI and combines them into a
169   * single
170   * Predicate using Predicate.and() - all filters must match (AND logic)
171   */
172  private static Predicate<Employee> parseFilters(String[] args) {
173      Predicate<Employee> combined = emp -> true; // start: accept all
174
175      for (int i = 0; i < args.length - 1; i++) {
176          if ("--filter".equals(args[i])) {
177              try {
178                  Predicate<Employee> next = FilterParser.parse(args[i +
179 1]);
180                  combined = combined.and(next);
181              } catch (IllegalArgumentException e) {
182                  System.err.println("Error in filter '" + args[i + 1] +
183                      "': " + e.getMessage());
184                  System.err.println("Run with -h or --help for usage
information.");
185                  System.exit(1);
186              }
187          }
188      }
189  }

```

```

185         }
186     }
187     return combined;
188 }
189
190 /** Prints the active filters, or "(none)" if no --filter was given. */
191 private static void printFilters(String[] args) {
192     boolean any = false;
193     for (int i = 0; i < args.length - 1; i++) {
194         if ("--filter".equals(args[i])) {
195             System.out.println("    " + args[i + 1]);
196             any = true;
197         }
198     }
199     if (!any) System.out.println("    (none - showing all employees)");
200 }
201 }

```

Sample Output

No filters

```

> java EmployeeProcessor
=== Active filters ===
(none - showing all employees)

=== Name, Department ===
Alice, Engineering
Bob, Marketing
Carol, Engineering
David, HR
Eve, Marketing
Frank, Finance

=== Average Salary ===
$73000.00

```

Filter: age above 30 and salary at least \$70,000

```

> java EmployeeProcessor --filter age>30 --filter salary>=70000
=== Active filters ===
age>30
salary>=70000

=== Name, Department ===
Alice, Engineering
Carol, Engineering
Frank, Finance

=== Average Salary ===

```

```
$91000.00
```

Filter: department contains “ing”

```
> java EmployeeProcessor --filter department~ing
=== Active filters ===
department~ing

=== Name, Department ===
Alice, Engineering
Carol, Engineering
Frank, Finance

=== Average Salary ===
$91000.00
```

Help output

```
> java EmployeeProcessor --help
Usage:
java EmployeeProcessor [-i file] [--filter expression] ...

Options:
-h, --help           Show this help message
-?, --help           Same as above
-i, --infile file    Read employees from a CSV file instead of
the built-in dataset
--filter expr        Filter employees by a field expression (
repeatable)

Filter expressions:
field operator value

Fields:      name, age, department, salary
Operators:  =    exact match (all fields)
~    contains, case-insensitive (name, department)
>    greater than (age, salary)
<    less than (age, salary)
>=   greater than or equal (age, salary)
<=   less than or equal (age, salary)

Multiple --filter flags are combined with AND logic.
```

Error handling

```
> java EmployeeProcessor --filter age>abc
```

```
Error in filter 'age>abc': 'abc' is not a valid number for field '
age'
Run with -h or --help for usage information.

> java EmployeeProcessor --badflag
Error: Unknown option '--badFlag'
Run with -h or --help for usage information.
```


References

- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java concurrency in practice*. Addison-Wesley Professional.
- Liang, Y. D. (2020). *Introduction to java programming and data structures* (12th ed.). Pearson.
- Urma, R.-G., Fusco, M., & Mycroft, A. (2018). *Modern java in action*. Manning Publications.